



RAPPORT DE PROJET

Mise en Cadre de Texte

Implémentation en C et en Langage Assembleur RISC-V

Architecture des Ordinateurs

2025 – 2026

BOUFERGUENE Léa
BENMEZIANE Khiredine
MAIN3

1. Introduction

Ce projet a pour objectif de réaliser un programme capable d'encadrer automatiquement le contenu d'un fichier texte à l'aide de caractères "*", en respectant une largeur maximale de ligne définie par l'utilisateur. Le texte peut être aligné selon trois modes au choix : aligné à gauche, centré, ou aligné à droite.

Le projet a été développé en deux phases : d'abord une implémentation en langage C servant de prototype et de référence algorithmique, puis une traduction complète en langage assembleur RISC-V, conformément aux objectifs du cours d'Architecture des Ordinateurs.

2. Description Globale du Programme

Le programme lit un fichier texte nommé «test», affiche un menu à l'utilisateur pour choisir un mode d'alignement parmi les 3 proposés ainsi que la largeur maximale par ligne, puis génère un fichier de sortie «test_contour» contenant le texte encadré de caractères "*".

Exemple : On choisit ici une largeur de 5 avec le texte «khiro lea», le résultat attendu est :

Contour gauche :	Contour milieu :	Contour droite :
*****	*****	*****
khiro	*khiro*	*khiro*
*lea *	* lea *	* lea*
*****	*****	*****

3. Explication du Code C

3.1 Structure du programme

La fonction main lit un fichier texte caractère par caractère, puis appelle les trois fonctions d'encadrement en passant une largeur fixe max_ligne = 5.

3.2 Les trois modes d'encadrement

- **contour (gauche)** : écrit directement dans le buffer. Gère les retours à la ligne, les espaces en début de ligne (ignorés), et les mots trop longs (renvoyés à la ligne suivante, ou erreur fatale si le mot dépasse la largeur).
- **contour_milieu (centré)** : accumule d'abord chaque ligne dans un tampon temporaire, puis calcule les espaces gauche/droite pour centrer le texte, en distribuant le pixel impair à droite.
- **contour_droite (droit)** : même logique de tampon, mais tous les espaces de remplissage sont placés à gauche.

3.3 Fonctions utilitaires

Trois fonctions de support : lire_mot (lit un mot depuis le fichier), ecrire_mot (copie un mot dans le buffer), et ecrire_n_fois_le_caractere (récupère un caractère n fois).

4. Explication du Code en Langage Assembleur RISC-V

Le fichier `fichier_final.asm` est la traduction complète du programme C en RISC-V 32 bits, exécutable dans RARS. Chaque fonction C possède un équivalent assembleur respectant strictement les conventions d'appel RISC-V.

4.1 Segment `.data`

Le segment `.data` déclare toutes les données statiques du programme :

- Chemins de fichiers : `path` ("test") et `path_output` ("test_contour").
- Messages du menu (`choix1`, `choix2`, `choix3`, `fleche`, `mauvais_choix`, `demande_nb_max`, `erreur_message`).
- Buffers : `chaine` (5000 octets pour le contenu lu), `buff` (5000 octets pour le résultat), `mot_temp` (100 octets, tampon intermédiaire), `ligne` (100 octets, utilisé dans les modes milieu et droite).
- Variable `taille_mot` (entier 32 bits), servant de variable de sortie par référence pour `lire_mot`.

4.2 La fonction `main`

Le point d'entrée `main` exécute la même logique que le `main` C :

- Ouverture du fichier (ecall 1024 — `open`, mode 0 = lecture) puis appel à la fonction `lecture`.
- Vérification de la terminaison de la chaîne : ajout de `'\n'` et `'\0'` si nécessaire.
- Affichage du menu (ecall 4 — `print_string`) et lecture du choix de l'utilisateur (ecall 5 — `read_int`) stocké dans `s0`.
- Lecture du nombre maximum de caractères par ligne (ecall 5), stocké dans `a2`.
- Branchement vers `contour`, `contour_milieu` ou `contour_droite` selon `s0`, ou affichage d'un message d'erreur si choix invalide.
- En cas de succès (`a0 == 1`) : écriture du buffer dans le fichier de sortie via `ecriture`. Terminaison par ecall 10.

4.3 Fonctions utilitaires

`ecrire_mot`

`ecrire_mot` est la traduction la plus directe du code C : elle sauvegarde 3 registres sur la pile (`s0`, `s1` et `ra`), soit 12 octets, puis copie simplement `a2` caractères depuis `mot` vers `buff`, un octet à la fois grâce aux instructions `lb` (load byte) et `sb` (store byte). Une fois la copie terminée, les registres sont restaurés et la fonction retournée.

`lire_mot`

Cette fonction sauvegarde 7 registres sur la pile (s0–s5 et ra), ce qui correspond exactement aux 28 octets alloués. La boucle avance caractère par caractère dans le fichier et s'arrête dès qu'elle rencontre un espace, un saut de ligne, un caractère nul ou un pointeur NULL — c'est-à-dire tout ce qui marque la fin d'un mot. Une fois le mot entièrement lu, sa taille est écrite en mémoire à l'adresse fournie par a2 via l'instruction sw, ce qui correspond au passage par référence du paramètre *int t en C*

ecrire_n_fois_le_caratere

L'idée est simple : plutôt que de répéter le même bloc d'écriture à plusieurs endroits du code (pour les bordures d'étoiles ou les espaces de remplissage), on regroupe cette logique dans une seule fonction qui écrit un caractère donné autant de fois qu'on le souhaite dans le buffer. Cela rend le code assembleur plus lisible et plus facile à maintenir.

lecture

Lit le fichier par blocs de 10 caractères (ecall 63 — read) et accumule dans chaine, en boucle jusqu'à ce que l'ecall retourne 0. Ferme ensuite le fichier (ecall 57 — close) et retourne le total de caractères lus dans a0.

écriture

Écrit le buffer de sortie dans le fichier caractère par caractère (ecall 64 — write), jusqu'à '\0'. Ferme le fichier via ecall 57.

4.4 Fonctions de contour

Les trois fonctions assembleur de contour (contour, contour_milieu, contour_droite) sont des traductions fidèles des fonctions C. Elles partagent la même structure générale et retournent 1 en cas de succès, 0 en cas d'erreur si le mot est trop long par exemple.

Structure commune :

- Prologue : On sauvegarde 11 registres (s2–s11 et ra) sur une pile de 120 octets. Les 120 octets ne correspondent pas uniquement aux 11 registres (ils n'en nécessiteraient que 44) : les emplacements restants sont réservés pour sauvegarder temporairement les registres a0, a1 et a2 à l'intérieur de la boucle, avant chaque appel de sous-fonction, afin de ne pas perdre les pointeurs actifs sur file, buff et max_taille_ligne (contour_milieu et contour_droite sauvegardent 13 registres (s0 et s1 en plus) parce qu'ils ont besoin de variables supplémentaires pour gérer ligne et taille_ligne).
- Initialisation des constantes dans les registres : s2='*', s3='\n', s4=' ', s7=0 (nb_caractères_écrits), s8='\0'.
- Appel à écrire_n_fois_le_caratere pour la bordure supérieure.
- Boucle while avec structure if/else_if.
- Une fois la boucle terminée, la bordure inférieure est écrite de la même manière que la supérieure, via un appel à écrire_n_fois_le_caratere (max_taille_ligne + 2 étoiles), suivi d'un saut de ligne '\n' et du caractère de fin de chaîne '\0'. La valeur de retour 1 (succès) est ensuite placée dans a0, puis les 11 registres sauvegardés sont restaurés depuis la pile avant le ret..

Différences :

- **contour (gauche) :** Contrairement aux deux autres modes, les caractères sont écrits directement dans buff au fur et à mesure de la lecture, sans passer par une variable intermédiaire. Les espaces de remplissage sont simplement ajoutés à droite du texte pour compléter la ligne jusqu'à la bordure.
- **contour_milieu :** Les caractères sont d'abord accumulés dans la variable ligne, puis l'espace disponible est divisé en deux pour centrer le texte; la moitié gauche est écrite avant le texte, la moitié droite après, le reste éventuel (en cas de nombre impair d'espaces) étant ajouté à droite. En assembleur, cette division est réalisée efficacement par un décalage de bits vers la droite (srli), et le reste est récupéré avec un ET bit à bit (andi).
- **contour_droite :** accumulation dans ligne, tous les espaces à gauche. Un bloc supplémentaire retire les espaces terminaux de ligne avant chaque écriture.

Il est à noter que comme les registres a0, a1, a2 servent à la fois de pointeurs actifs et d'arguments aux sous-fonctions, ils sont systématiquement sauvegardés sur la pile avant chaque call et restaurés avec mise à jour de l'offset (add a1, a1, a2) après.

5. Organisation et Conventions Adoptées

Plusieurs choix techniques ont structuré le développement :

- Conventions RISC-V : a0–a7 pour arguments et valeur de retour, s0–s11 pour variables locales persistantes, ra pour l'adresse de retour.
- Gestion de la pile : prologue (addi sp, sp, -N / sw) et épilogue symétrique (lw / addi sp, sp, N) systématiques dans chaque fonction.
- Sauvegarde temporaire de a0, a1, a2 autour des appels de sous-fonctions, nécessaire car ces registres sont utilisés comme pointeurs actifs dans les boucles.
- Factorisation via `ecrire_n_fois_le_caratere` pour éviter la répétition de blocs d'écriture identiques.
- Normalisation du fichier d'entrée : ajout automatique de '\n' final si absent, garantissant un comportement correct de toutes les fonctions de contour.

6. Difficultés rencontrées

Léa : Pour ma part, j'ai pris l'habitude de commencer par coder en C avant de passer à l'assembleur, ce qui me permet d'avoir une vision claire de la structure. Le code C nous a pris beaucoup de temps car il fallait gérer de nombreuses variables, des pointeurs et une multitude de cas particuliers, mais il nous a servi de base solide pour la suite. Khireddine avait déjà de solides bases en assembleur, ce qui a beaucoup aidé. De mon côté, la plus grande difficulté a été de gérer un grand nombre de registres simultanément, car la moindre erreur provoque des comportements difficiles à déboguer. Ce projet m'a néanmoins permis de découvrir de nouvelles instructions RISC-V et de pratiquer à une bien plus grande échelle ce que je connaissais déjà.

Khiredine : Pour moi, quand nous avons fini la traduction du code C en assembleur, nous étions sûrs que ça devrait marcher mais les deux fonctions `contour_milieu` et `contour_droite` ne marchent pas et ça met une boucle infinie. Après de longues heures à débbugger, je me suis rendu compte que j'ai utilisé les registres `t0` et `t1` à un moment qui n'étaient pas préservés après un appel de fonction (merci au créateur de RARS qui permet de voir les valeurs de registres, sinon je saurai jamais où est l'erreur). Et donc pour éviter les sauvegardes dans la pile avant l'appel, j'ai remplacé `t0` par `a5` et `t1` par `a6`. Ce projet est fun à implémenter mais néanmoins ça montre qu'un programme assembleur demande beaucoup de concentration et de mémoire pour un humain car il faut s'en souvenir de la représentation de chaque registre sinon on risque de se perdre facilement.

6. Conclusion

Ce projet nous a permis de concevoir et implémenter un programme complet de mise en forme de texte, d'abord en C pour valider la logique algorithmique des trois modes d'alignement, puis en assembleur RISC-V pour en maîtriser la traduction bas niveau.

Les principaux défis ont été la gestion simultanée de plusieurs pointeurs actifs dans les boucles, la sauvegarde rigoureuse des registres autour de chaque appel de sous-fonction, et la mise en œuvre du calcul de centrage (division par 2 par décalage de bits) pour le mode milieu.

Le résultat final est un programme assembleur fonctionnel proposant les trois modes d'alignement, capable de lire et écrire des fichiers via des appels système, et gérant correctement les cas d'erreur, tout en respectant scrupuleusement les conventions d'appel RISC-V.